HTML-CSS-04 第4回_まとめ

第4回の目標

- **よいCSSの考え方を知る**
  - **予測しやすい** →他の人がコードを見たときに、記述がどのように影響しているかわかりやすいように書く
  - **再利用しやすい** →使いまわせるパーツのスタイルはまとめる
  - **保守しやすい** →修正・変更した時にわかりやすい書き方にする
  - **拡張しやすい** →追加変更があったときに、最低限のコストで作れるようにする
- **CSS設計に基づいた、クラス名のつけ方を知る**
  - 原則は英語
  - 基本的な命名規則はBEM
  - ハイフン、接頭辞を正しく使う
- **CSS変数を使えるようになる**
- **リファクタリングをおこなう**
  - CSS設計の考え方やルールに基づいて、外部の見た目を変化させずに，内部構造を整理するためにコードを書き換えていく作業
    - CSS設計の命名規則を設定する
    - 違うクラス名だが見た目が同じ要素は、クラス名を統一する
    - カラーコードやコンテンツ幅、z-indexなどをCSS変数を用いて管理する

カスタムプロパティ

- CSS内で「変数」を使った記述の方法。
- よく使用する「値」を変数の中に入れておくことで、何度も同じ記述を書かなくて済む
- 変数を定義するときは「`:root`（ルートセレクタ）」の中で変数を作る
  - 変数の名前を決めるときは、先頭に「`--`（ハイフン）」を 2 つ添える
- 変数を呼び出すときは、CSSの値の箇所で「`var()`」を使用
  - 書き方：`プロパティ名: var(変数名);`（例：`color: var(--primary-color);`）
  - `var`の2文字目を「e」と間違えやすいので注意
  - 変数が使える場所で`--`と打つと、予測変換で変数名が出てくる

以下 2 つの読み込み結果は同じ

```
.sample {
  color: #ffbb1e;
```

```
}
```

```
:root {
  --primary-color: #ffbb1e;
  --accent-color: #43cfe8;
}

.sample {
  color: var(--primary-color);
}
```

calc関数

- CSSを記述する際に「計算式」を使うことができる関数

使用例

横幅が100%の要素を３等分するようにスタイルを記述するとき

△

```
.sample {
  width: 33.33333%;
}
```

◎

```
.sample {
  width: calc(100% / 3);
}
```

- calc()を使って値を計算しよう

画面幅に合わせて残りの幅を使う

```
/* サイドは260pxで固定、メインは残りの幅いっぱい */
.calc_main {
```

```
  width: calc(100% - 260px);
}
```

- `100%` は「画面幅に応じた親要素の幅いっぱい」を表す
- `calc(1080px - 260px)` と書くと820pxで固定されてしまう。親の `max-width` が1080pxでも、画面が狭ければ親は1080pxより小さくなるので、`100%` を使う

【＋α】Sass（SCSS）

- CSSをより効率的に書ける言語。CSS変数を調べるとよく出てくる
- 授業では扱わないが、ZeroPlusポータルの特別講義で学べる

CSS変数で管理する値（演習）

繰り返し出てくる値、特に**色・コンテンツ幅・z-index**はCSS変数にまとめて一覧で管理する。

```
:root {
  /* 色管理用の変数 */
  --white-color: #fff;
  --black-color: #333;
  --gray-color: #e8e8e8;
  --gray-color02: #f7f7f7;
  --gray-color03: #aaa;
  --primary-color: #008db7;
}

:root {
  /* コンテンツ幅管理用の変数 */
  --content-width-sm: 760px;
  --content-width: 840px;
}

:root {
  /* z-index管理用の変数 */
  --z-index-back: -1;
  --z-index-default: 1;
  --z-index-header: 100;
  --z-index-menu: 150;
```

```
  --z-index-modal: 200;
}
```

CSS変数に置き換えるメリット

- `z-index`を一覧で管理することで、重なり順が予測しやすくなる
- 色やコンテンツ幅に変更があったときに、`:root`内の記述を書き換えれば一括で変更できる
- 色の変数を作るときは、見た目（`--blue`）ではなく**役割の名前**（`--primary-color`）にする（後述の「デザイン依存の命名を避ける」参照）
    - 白・黒・グレーなど、ベースとして使う色は色の名前でよい
    - メインカラーやアクセントカラーは、サイトによって違ったり、あとで変わったりするので役割の名前にする
- 変数は、色に限らずフォントサイズや幅など、サイト内で使う値そのものを一元管理できる

スニペット登録と変数登録の違い

	役割
スニペット登録	入力を楽にするためのショートカット
変数登録	CSSの中で値を管理・再利用する仕組み。メインカラーを一括で変えたいときなどに対応できる
- CSS変数について	

コンテンツ幅をまとめる（l_container）

ページごとに書いていたコンテンツ幅のスタイルを、レイアウト用のクラスにまとめる。 - 「左右16pxの余白」「要素を中央寄せ」「横幅の制限」など、共通するレイアウトを一括管理できるようにクラス名をまとめておく - `l_container`は次回以降の授業でもよく出てくる

```
/* 共通する見た目を管理するクラス */
.l_container,
.l_container-sm {
  padding: 0 16px;
  margin-left: auto;
  margin-right: auto;
}
```

```
/* 個別ページのコンテンツの幅 */
.l_container-sm {
  max-width: calc(var(--content-width-sm) + 32px);
}

/* トップページとカテゴリーページのコンテンツの幅 */
.l_container {
  max-width: calc(var(--content-width) + 32px);
}
```

- `+ 32px` は左右の `padding`（16px × 2）の分。`padding` を含めた幅にしておくことで、中身の幅がちょうど `--content-width` になる
- `calc()` の中でも `var()` でCSS変数を使える
- HTML側は `class="l_container m_posts"` のように、既存のクラスに追加して使う
- メリット
  - 再利用しやすい：開発中に同じ記述を繰り返さなくて済む
  - 保守しやすい：コンテンツ幅の変更があったときに、ここの記述を書き換えれば一括で変更できる
  - 拡張しやすい：ページの追加があったときに、コンテンツ幅をそのまま使える

命名規則

CSS設計とは

- 「良いCSS」を書くための考え方に基づいたルールのこと
- BEM・FLOCSS・PRECSSなど色々なルールがあり、絶対的な正解はない
- ZeroPlusでは「BEM＋接頭辞」の独自ルールを採用している
- 大きなサイトはファイル数が多く、何年も使われる。CSS設計がしっかりしているとサイトが長生きする
- クラス名は基本は英単語でつけるが、英語だと分かりにくい・長くなるときは日本語ローマ字にすることもある（例：特商法 → `tokusyoho`）
- イメージがつかないときは、ChatGPTに「CSS設計の4原則（予測しやすい・再利用しやすい・保守しやすい・拡張しやすい）について、予測しにくいクラス名や再利用しにくいクラス名の具体例を教えて」と聞いてみるのもよい

BEM

- Block（ブロック）, Element（エレメント）, Modifier（モディファイア）の頭文字から。

- 書式：`Block` `_` `Element` `__` `Modifire`

    - 意味：`どこの` `_` `何か` `__` `役割`
    - Modifierはなくてもよい
- Modifierは、形は同じでパーツにしたいが、色やフォントサイズが少し違うときに使う
    - 例：`pagination_numbers__current`（現在のページだけ背景色を変える、という特定の役割）

ハイフンとアンダーバーの使い分け

「〜の」がつくところをアンダーバーにするイメージ。

日本語	クラス名
ゼロプラス講師	`zeroplus-teacher`
ゼロプラス**の**講師	`zeroplus_teacher`
ゼロプラス講師**の**めい先生	`zeroplus-teacher_mei-sensei`

Block・Element・Modifierの中身

	指すもの	つなぎ方	例
Block	ある程度大きい1つのかたまり	—	`header`、`footer`、`header-nav`
Element	Blockの中の「何か」	`_`（アンダーバー1つ）	`header-logo_link`の`link`
Modifier	一部のスタイルだけ変えたいとき、バージョン違いを作りたいときの役割	`__`（アンダーバー2つ）	`pagination_numbers__current`

- Elementを使わないこともある（例：`l_header`は接頭辞とBlockだけ）
- Modifierのクラスには、共通スタイルを持つクラスも一緒につけることが多い（例：`pagination_numbers` `pagination_numbers__current`）
- どこをハイフンでつなぎ、どこにアンダーバー（「の」）を入れるかに明確な基準はない
    - `header-nav_list`（ヘッダーナビのリスト）でも、`header_nav-list`（ヘッダーのナビリスト）でもよい
    - ただし**サイト内で統一する。**`header_nav-list`と`header-nav_item`が混ざると保守性が下がる
- クラス名は「誰が見ても何を指しているか分かる」ことを目指す。略すと分からなくなるものは無理に略さない（`txt`・`btn`などはよく使われる略）

BlockとElementは1つずつ

クラス名	判定	理由
`header_logo_link`	×	ヘッダー（Block）のロゴ（Element）のリンク（Element）で、Elementが2つある
`header-logo_link`	○	ヘッダーロゴ（Block）のリンク（Element）
`header_logo-link`	○	ヘッダー（Block）のロゴリンク（Element）

【BEMで書かない場合のコード】

```
<header class="header">
  <h1 class="header-logo">
    ZeroPlus <span class="header-logo-primary">Ad</span>Sense
  </h1>
</header>
```

【BEMに直したもの】

```
<header class="header">
  <h1 class="header_logo">
    ZeroPlus <span class="header_logo__primary">Ad</span>Sense
  </h1>
</header>
```

ここでは、 - `Block`＝`header` - `Element`＝`logo` - `Modifire`＝`primary`

`Block`でよく使うフレーズ

- `nav`
  - ナビゲーションで用いる。
- `post(s)`
  - blogサイトの投稿・記事に用いる。
- `unit(s)`
  - 複数の要素をまとめているものに用いる。
- `list`
  - リストに用いる。
- `item(s)`
  - 複数あるうちの一つとして用いる。
- `btn`
  - ボタンに用いられる。buttonの慣例的な省略方法。

- `kv`
  - ページのファーストビューの画像とキャッチコピーなどをまとめたものに用いる。key visualの慣例的な省略方法。

`Element`でよく使うフレーズ

- `cat(s)`
  - カテゴリーに用いられる。categoryの慣例的な省略方法。
- `content(s)`
  - 内容に関する要素を囲う時に用いる。
- `meta`
  - 日付やカテゴリーなどの情報を扱う時に用いる。
- `link`
  - リンクに用いられる。aタグによく使う。
- `title`
  - タイトルに用いられる。単語であることが多い。
- `heading`
  - 見出しに用いられる。文章であることが多い。
- `desc`
  - 説明文や概要の文章に用いられる。descriptionの慣例的な省略方法。
- `txt`
  - テキストに用いられる。抽象度が高いため便利だが注意が必要。
- `thumb`
  - サムネイルに用いる。thumbnailを省略した形。
- `icon`
  - アイコン画像に用いる。
- `logo`
  - ロゴ画像に用いる。
- `img`
  - それ以外の画像に用いる。汎用的で便利。
- `date`
  - 日付に用いられる。
- `copyright`
  - 著作権表記に用いられる。

`Modifier`でよく使うフレーズ

- `sm`
  - 比較的小さいもの、またはスマホサイズの画面幅の表示に用いられる。smallの慣例的な省略方法。
- `md`

    - 比較的中くらいの大きさ、またはタブレットサイズ以上の画面幅の表示に用いられる。mediumの慣例的な省略方法。
- `lg`
    - 比較的大きいもの、またはパソコンサイズ以上の画面幅の表示に用いられる。largeの慣例的な省略方法。
- `primary`
    - サイトで重要なものに用いられる。色や文字によく用いる。
- `sub`
    - サイトで補助的に使用するものに用いられる。

接頭辞

BEMで書いたclass名の前につける文字のこと

接頭辞	役割
`l_class-name`	汎用的に使えるレイアウトにつかう。sectionレイアウトなど。headerやfooterは便宜上一つのレイアウトとして扱う。
`m_class-name`	繰り返し汎用的に使えるモジュール（コンポーネントと似たモノ）につかう。
`u_class-name`	ユーティリティクラス（単一のスタイル・役割を持つclass）につかう。
`page_class-name`	そのページ固有の部分につかう。ページ名を接頭辞にする（`top_`・`single_`・`contact_`など）。
`js_class-name`	jsセレクタにつかう。JavaScriptで操作する要素だとひと目でわかるようにする（例：`js_hamburger`）。

- ☝🏻 接頭辞の考え方の参考元：PRECSS（layout）／PRECSS（module）

接頭辞をつける理由

- BEMだけだと、どこのページで使われているかが分からない
- 接頭辞があると、どこのページにあるか、どんな役割・影響を持つかのヒントになる
    - 例：`l_`や`m_`がついていたら「ここを変えると、いろんなページやパーツに影響が出るかも」と注意できる

接頭辞の注意点

- 接頭辞は**1つのクラス名に1つだけ**
- **すべてのクラス名に1つ**接頭辞をつける
- `l_`は、BEMのBlockが`header`・`footer`・`main`・`container`のものにつけるイメージ（例：`l_header`、`l_container-sm`）

- `u_` は、1つの役割しか持たず、汎用的に使える便利なクラス。今は「レスポンシブ時に改行の有無を操作するクラス」くらいの理解でよい（出番は少ない）

クラス名	判定
`top_m_post`	✕ 接頭辞が2つ
`top_post` と `m_post` の2つのクラスをつける	○ モジュールの役割とトップページ独自のスタイルを両方持たせられる

- ハンバーガーメニューは、ヘッダーの中にあっても `m_` にする。必ずしもヘッダーの中にあるとは限らないため（`l_` でも間違いではない）
- `top_` がついているクラスはトップページでしか使わないので、気軽に修正できる。`m_` は複数ページで使っているので、修正すると別の場所が崩れる可能性がある

接頭辞の覚え方

最初から完璧につける必要はない。 1. とりあえずHTMLを作る 2.「あれ？これ使い回してない？」と気づく 3.「じゃあ `m_` をつけよう」と直す

作ってみて、役割に照らし合わせて修正を繰り返すうちに覚えていく。

授業での接頭辞の使い分け（演習）

元のクラス名	書き換え後	理由
`header` / `main` / `footer`	`l_header` / `l_main` / `l_footer`	ページの大きな枠（レイアウト）なので `l_`
`top-title` / `category-title`	`m_page-title`	複数ページで同じ見た目なので1つのモジュールに統一
`post` / `pagination`	`m_post` / `m_pagination`	複数ページで使い回す部品なので `m_`
`category-title_break`（改行用の `br`）	`u_md-dn`	「md（中間幅）以上でdisplay: none」という単一の役割なので `u_`
`single_〇〇`	そのまま	個別ページにしか登場しないので、ページ名の接頭辞

```
/* トップページのタイトルだけ色を変える → Modifierで差分を追加 */
.m_page-title__primary-color {
  color: var(--primary-color);
}
```

```
/* 中間幅（768px）以上でbrを消して改行をなくす */
@media screen and (min-width: 768px) {
  .u_md-dn {
    display: none;
  }
}
```

- HTMLのクラス名を置換するときは、タグ名まで変わってしまわないように注意する
- 【＋α】命名の参考になる英単語

- **l**：layout（レイアウト）
    - 共通化できる、大きな「枠」専用のグループ
    - 原則として、`width`や`margin`や`padding`などのようなレイアウトに関わるスタイリングのみ行う。

  ``` /* コンテンツごとの上下の余白の例 */ .l_contents-wrapper { padding: 120px 0; }

  /* コンテンツ幅の例 */ .l_container { margin-left: auto; margin-right: auto; max-width: 1200px; width: 92%; } ```

- **m**：module（モジュール）
    - どこでも使いまわせるパーツのグループ
    - 基本的に、レイアウトに関わるスタイリングは行わない。
        - 拡張性を考えると、レイアウトに関することはMixを使ったり、親要素に依存すると良い `<div class="single_main_meta"> <div class="m_meta"> <time class="m_meta_date">2021.06.05</time> <a href="category.html" class="m_meta_cat">SEO</a> </div> <!-- /.m_meta --> </div>` ``` /* モジュールの例 */ .m_meta { display: flex; justify-content: space-between; align-items: center; } .m_meta_date { color: #aaa; font-size: 12px; font-weight: bold; } .m_meta_cat { color: #008db7; font-size: 12px; font-weight: bold; width: 116px; height: 25px; display: flex; align-items: center; justify-content: center; border: 1px solid #008db7; border-radius: 5px; }

  /* レイアウトを分担させている例 */ .single_main_meta { margin-top: 16px; } ```

- **u**：utility（ユーティリティ）
    - class名がそのまま、CSSのプロパティ名と値を表現しているもののグループ
    - 名前の付け方はEmmetを参考にすることが多い。
    - Web制作では補助的に使われることがある

  例：上に10pxのmarginを付けたい箇所に使い回したいときのクラス名 `/* 「.u_プロパティ-値」の順で指定している */ .u_mt-10 { margin-top: 10px; }` 別箇所で使う際にも使えるような書き方になっている。
  ```

BEM以外に使う命名規則

- 2単語以上をつなげる場合はハイフン (-)
    - ex.) `.class-name`
- モジュール・レイアウト・ユーティリティクラスに関しては接頭辞をつける
    - ex.) `.m_class-name`
- ページ独自のスタイルはページの名字を接頭辞とすることとする。
    - ex.) `.top_block_element__modifier`
- 汎用的に使用可能な名前
    - `wrapper`
        - 意味
            - 包むもの
            - 外側の入れ物
        - 使う場面
            - コンテンツ全体を囲む最も外側の要素
            - 余白や幅の制御をするためのコンテナ
        - よく使う用途
            - ページ全体
            - セクション全体
            - コンポーネント全体
    - `inner`
        - 意味
            - 内側
        - 使う場面
            - `wrapper`の内側に配置し、実際のコンテンツを包む要素
            - 横幅や左右の余白を調整するためのコンテナ
        - よく使う用途
            - セクション内のレイアウト
            - コンテンツの横幅制御
            - 中央寄せ
    - `header`
        - 意味
            - 先頭部分
            - 見出し部分

        - 使う場面
            - ページやコンポーネントの上部
            - タイトルやナビゲーションを配置する部分
        - よく使う用途
            - ページヘッダー
            - カードヘッダー
            - モーダルヘッダー

    - `body`

        - 意味
            - 本体
            - メインコンテンツ
        - 使う場面
            - コンポーネントの中心となる内容を配置する部分
        - よく使う用途
            - 説明文
            - 画像
            - フォーム
            - 一覧

    - `footer`

        - 意味
            - 末尾部分
            - 終端部分
        - 使う場面
            - ページやコンポーネントの下部
            - 補足情報や操作ボタンを配置する部分
        - よく使う用途
            - コピーライト
            - ボタン
            - リンク
            - 投稿日時

- 単語を省略する場合

    - `cat(s)`

        - 正式名称
            - `category`
            - `categories`

    - 意味
        - カテゴリー
        - 分類
    - 使う場面
        - 記事や商品のカテゴリ分け
    - よく使う用途
        - カテゴリー一覧
        - タグ表示

- `col`
    - 正式名称
        - `column`
    - 意味
        - 列
    - 使う場面
        - グリッドレイアウト
        - テーブル
    - よく使う用途
        - レスポンシブレイアウト
        - 複数カラム表示

- `lv`
    - 正式名称
        - `level`
    - 意味
        - レベル
        - 階層
    - 使う場面
        - メニューや見出しの階層管理
    - よく使う用途
        - `menu-lv1`
        - `heading-lv2`

- `desc`
    - 正式名称
        - `description`
    - 意味
        - 説明

        - 説明文
    - 使う場面
        - 商品や記事の概要文
    - よく使う用途
        - 商品説明
        - カード説明
        - キャプション

- `btn`
    - 正式名称
        - `button`
    - 意味
        - ボタン
    - 使う場面
        - クリック操作を行う要素
    - よく使う用途
        - 送信ボタン
        - リンクボタン
        - CTAボタン

- `img`
    - 正式名称
        - `image`
    - 意味
        - 画像
    - 使う場面
        - 画像を表示する要素
    - よく使う用途
        - 商品画像
        - アイキャッチ画像
        - プロフィール画像

- `thumb`
    - 正式名称
        - `thumbnail`
    - 意味
        - サムネイル画像
    - 使う場面

        - 一覧表示用の小さい画像
    - よく使う用途
        - ブログ一覧
        - 商品一覧
        - 動画一覧

- `num`
    - 正式名称
        - `number`
    - 意味
        - 数字
        - 番号
    - 使う場面
        - 数値を表示する要素
    - よく使う用途
        - ランキング
        - ステップ番号
        - 価格表示

- `txt`
    - 正式名称
        - `text`
    - 意味
        - テキスト
        - 本文
    - 使う場面
        - 文章を表示する要素
    - よく使う用途
        - 本文
        - 説明文
        - コメント

- `sm`
    - 正式名称
        - `small`
    - 意味
        - 小サイズ
    - 使う場面

        - サイズ指定
    - よく使う用途
        - `btn-sm`
        - `text-sm`

- `md`
    - 正式名称
        - `medium`
    - 意味
        - 中サイズ
    - 使う場面
        - サイズ指定
    - よく使う用途
        - `btn-md`
        - `text-md`

- `lg`
    - 正式名称
        - `large`
    - 意味
        - 大サイズ
    - 使う場面
        - サイズ指定
    - よく使う用途
        - `btn-lg`
        - `text-lg`

- `kv`
    - 正式名称
        - `key visual`
    - 意味
        - キービジュアル
        - メインビジュアル
    - 使う場面
        - ページのファーストビュー
    - よく使う用途
        - LPのメイン画像
        - コーポレートサイトのトップ画像

- `cta`
    - 正式名称
        - `call to action`
    - 意味
        - 行動喚起
    - 使う場面
        - ユーザーに行動を促す要素
    - よく使う用途
        - お問い合わせボタン
        - 資料請求ボタン
        - 購入ボタン
        - 無料登録ボタン

その他意識すること

詳細度をあげないこと

良いCSSを書くためには、セレクタの詳細度をコントロールする必要がある

例えば、次のようなスタイルの指定をむやみに行うと、破綻が起きたり改修段階で不要なコストがかかる。

```
/* 同じ「!important」でしか上書きができなくなる */
.example {
  color: red !important;
}

/* 上書きしづらい + 使いまわせない */
#about {
 /* #aboutに対して与えるスタイル */
  color: red;
}
```

基本的にはclassセレクタで同じ詳細度で書く。

どうしてもidセレクタを使わなければならない時は、次のようにして詳細度を上げることを回避

```
[id='about'] {
 /* #aboutに対して与えるスタイル */
```

```
  color: red;
}
```

デザイン依存の命名を避ける

命名方法で、次のように「見た目からつける」というのは避けた方が良い。

```
:root {
  --yellow-color: yellow;
  --blue-color: blue;
}

.class-name__color-red {
  color: red;
}
```

このように書いてしまうと、値が変わった時に名前まで変更の必要があり、保守性に欠ける。次のように「意味や役割からつける」方法をとるとよい。

```
/* 変数の命名も役割からつける */
:root {
  --primary-color: yellow;
  --accent-color: blue;
}

/* 警告用の色として扱う */
.class-name__warning {
  color: red;
}
```

リファクタリング → 改修してみる

リファクタリングとは、CSS設計の考え方やルールに基づいて、外部の見た目を変化させずに内部構造を整理するためにコードを書き換えていく作業。つまり「Webサイトの見た目を変えずに、綺麗なコードに書き換える」こと。

手順

1. CSS変数を使って、色・コンテンツ幅・z-indexを管理する
2. レイアウト（`l_`）でまとめる：`header`・`main`・`footer`・コンテンツ幅
3. モジュール（`m_`）でまとめる：ページタイトル・記事一覧・ページネーション
4. ユーティリティ（`u_`）でまとめる：改行の出し分けなど
5. ページ独自のスタイル（`single_`など）を整理する
6. 違うクラス名だが見た目が同じ要素は、クラス名を統一する

クラス名を一括で置き換える

VSCodeの検索・置換を使うと、まとめて書き換えられる。

対象	検索する文字	置き換える文字
HTMLのクラス名	`"header`（ダブルクォーテーションを含める）	`"l_header`
CSSのセレクタ	`.header`	`.l_header`

- `"`や`.`を含めて検索すると、関係ない文字列まで置き換えるのを防げる
- 左のパネルの虫メガネアイコンから検索すると、フォルダの中身を一括で検索・置換できる
- 1つのファイルだけか、フォルダ内のすべてのファイルかは、状況に応じて使い分ける

1ファイル内の置換手順 1. `Ctrl`（Mac：`Command`）＋`F`を押す 2. 検索窓の左の「>」を押す 3. 上段に置換前、下段に置換後の文字を入れる 4. 置換のアイコンを押す

- `"header`を`"header"`のように後ろにも`"`をつけると、クラスが複数ついている要素が置換されないので、頭だけにつける
- `header`だけで置換すると、`<header>`タグまで置換されてしまう。クラス名以外で使われていないか確認する
- 1つ置換したら、GoLiveの画面でこまめに確認する。崩れたまま進むと直すのが大変
- 間違えたら、直後なら`Ctrl`（`Command`）＋`Z`で戻す。時間が経っていたら、間違えた文字を上段、正しい文字を下段に入れて置換し直す
- 慣れると最初から接頭辞やCSS変数を使って書くが、最初は気にせず書いて、あとからリファクタリングしてもよい

改修してみる（修正依頼の想定）

共通化ができていると、スタイルを1か所変更するだけで全ページに効率的に反映される。 - メインカラーを`#ff6347`に変更したい → `:root`の`--primary-color`を書き換えるだけ - ヘッダーナビの背景色をフッターと同じにしたい → 同じ色の変数を指定し直すだけ - トップ・カテゴリーページのタイトルの`font-size`をSP 28px／PC 36pxにしたい → `.m_page-title`を1か所直すだけ

CSS設計の学び方

- CSS設計は奥が深いので、「理解を目指す」より「概要をざっくり知る」くらいの気持ちで学ぶ。Lesson5〜8の模範解答で少しずつ慣れればよい
- まずZeroPlusのルールに慣れ、慣れてきたら他の人の記事や書籍で知見を広げる（記事によってルールは違う）
- Webサイトは改修する前提で作ることが多い。メンテナンスしやすい「良いCSS」で書くと、改修の時間とコストを減らせる
- HTML・CSSには流行りがあるので、習得したあとも情報収集を続ける
- CSS設計において特に大切にしている思想
- 書籍：「CSS設計完全ガイド」（BEMとPRECSSを解説。ZeroPlusの接頭辞はPRECSSの影響が大きい）、モダンHTML&CSS 現場の新標準ガイド

本日のまとめ

1. よいCSSとは「予測しやすい・再利用しやすい・保守しやすい・拡張しやすい」CSSのこと
2. クラス名は「どこの何か」がわかるような命名にする（BEM：`block_element__modifier`）
3. 接頭辞（`l_`・`m_`・`u_`・ページ名・`js_`）は、そのクラス名がどんな役割なのかをひと目でわかるようにするために付ける
4. CSS変数を使うことで、色やコンテンツ幅、z-indexなどの管理がしやすくなる
5. リファクタリングは、Webサイトの見た目を変えずに綺麗なコードに書き換えること

参考記事

- CSS変数について
- calc()を使って値を計算しよう
- CSSにルールを設けてさらに保守性を高めよう part1
- CSSにルールを設けてさらに保守性を高めよう part2
- ZeroPlusでの命名規則
- 命名の参考になる英単語（Qiita）
- 命名の参考になる英単語（Qiita・授業チャットで紹介）
- PRECSS（layout）
- PRECSS（module）